High-Performance Computer Architecture Review
Chapters 1–6 with Integrated Cache Simulation Concepts

ECE 4100/6100

Spring 2026

Contents

Scope and Topic Map **7**

1 Introduction **9**
1.1 What Computer Architecture Is . . . 9
1.2 Core Methods: Parallelism, Speculation, and Prediction . . . 9
1.2.1 Speculation Without Prediction . . . 9
1.2.2 Speculation With Prediction . . . 10
1.3 Parallelism by Grain Size . . . 10
1.4 Flynn's Taxonomy . . . 10
1.4.1 SIMD vs. MIMD . . . 11
1.4.2 Where pipelining fits . . . 11
1.5 Vectors: Packed-Word SIMD vs. Pipelined Vector Units . . . 11
1.6 Common Chapter 1 Pitfalls . . . 12
1.7 Chapter 1 Summary . . . 12

2 Performance Evaluation **13**
2.1 Why Performance Evaluation Matters . . . 13
2.2 Throughput vs. Runtime . . . 13
2.3 CPU Runtime Equation . . . 13
2.4 RISC vs. CISC Through the Performance Equation . . . 14
2.5 Amdahl's Law . . . 14
2.6 Gustafson's Law and Why It Does Not Contradict Amdahl . . . 15
2.7 Combining Performance Numbers Correctly . . . 15
2.8 Workload Dependence and Benchmarking . . . 16
2.9 Chapter 2 Summary . . . 16

3 Cache Memories **17**
3.1 Memory Technology and the Hierarchy . . . 17
3.2 Typical Hierarchy Levels . . . 17
3.3 Locality: Why Caches Work . . . 17
3.4 Cache Geometry and Address Partitioning . . . 18
3.5 Direct-Mapped, Set-Associative, and Fully Associative . . . 19
3.6 Hit, Miss, and Average Access Time . . . 20
3.7 The Three C's of Misses . . . 20
3.8 Replacement Policies and Their Tradeoffs . . . 20
3.9 Write Policies . . . 20
3.10 Reducing Miss Ratio . . . 21

3.11 Reducing Miss Penalty . . . 22
3.12 Reducing Hit Time . . . 22
3.13 Virtual Memory Technology, Terminology, and Analogy to Caching . . . 23
3.14 Compiler and Data-Layout Techniques . . . 23
3.15 Chapter 3 Summary . . . 24

4 Pipelining **25**
4.1 Pipeline Motivation . . . 25
4.2 Five-Stage Example Pipeline . . . 25
4.3 Pipeline Speedup Calculation . . . 25
4.4 CPI Model and Stall Accounting . . . 26
4.5 Program Characteristics Plus Hazards Produce Stalls . . . 26
4.6 Structural Hazards . . . 26
4.7 Data Hazards . . . 26
4.7.1 Forwarding (Bypassing) . . . 27
4.7.2 Load-Use Stalls . . . 27
4.8 Control Hazards . . . 27
4.8.1 Delay Slots . . . 27
4.8.2 Squash Slots with Likely Bits . . . 28
4.9 Branch Target Buffer (BTB) . . . 28
4.9.1 BTB with Tags . . . 28
4.9.2 BTB without Tags . . . 28
4.9.3 Hardware Return Address Stack . . . 28
4.10 Whether-To Branch: Direction Prediction . . . 28
4.11 Timing-Diagram Interpretation (Appendix Reference) . . . 28
4.12 Chapter 4 Summary . . . 29

5 Integrated Review and Cache Simulation Concepts **31**
5.1 Chapter Purpose . . . 31
5.2 Homework Themes from Chapters 1 and 2 . . . 31
5.3 Homework Themes from Chapter 3 . . . 31
5.4 Chapter 4 Practice Problems (with Answers) . . . 32
5.5 Cache Simulation Conceptual Model . . . 32
5.6 Hierarchy Semantics in the Model . . . 32
5.7 Prefetching Concepts . . . 32
5.8 Statistics and AAT Interpretation . . . 33
5.9 Conceptual Tradeoffs Across Parameters . . . 33
5.10 Chapter 5 Summary . . . 33

6 Instruction-Level Parallelism and Superscalar **35**
6.1 Why CPI Can Go Below One . . . 35
6.2 Historical Motivation . . . 35
6.3 Superscalar Pipeline View . . . 36
6.4 Dependency Types and Why They Matter . . . 36
6.5 Fire/Complete Taxonomy . . . 37
6.6 Scoreboarding Intuition (FICO Style) . . . 38
6.7 Function Units and Latency . . . 38
6.8 Register Renaming as the Next Step . . . 38

6.9 Tomasulo's Algorithm 38
6.10 Result Buses and Physical Register Files 40
6.11 Speculation Without and With Prediction 40
6.12 Chapter 6 Summary 40

A Pipeline Timing-Diagram Interpretation Checklist 41
A.1 How to Use This Checklist 41

B Formula Compendium (Grouped and Labeled) 45

C Glossary of Core Definitions 49

Scope and Topic Map

How to Use This Text

Chapters 1 through 6 provide topic depth with definitions, equations, and worked examples. Chapter 5 provides integrated review and cache-simulation context, while Chapter 6 introduces instruction-level parallelism and superscalar design ideas.

Topic Coverage Map

Chapter 1: Introduction Coverage includes speculation with and without prediction, grain sizes of parallelism (bit, instruction, thread, and inter-program), Flynn taxonomy, SIMD versus MIMD, pipelining as temporal parallelism, and the distinction between packed-word SIMD and pipelined vector operations.

Chapter 2: Performance Evaluation Coverage includes throughput versus runtime, the CPU-time equation in terms of IC, CPI, and cycle time, RISC versus CISC tradeoffs, Amdahl's and Gustafson's laws, and correct use of arithmetic versus harmonic means.

Chapter 3: Cache Memories Coverage includes SRAM versus DRAM and bit-line delay, locality, hierarchy levels, virtual-memory terminology and its analogy to caching, average access time, the three C's of misses, cache geometry and construction, replacement and write policies, techniques to improve miss rate, miss penalty, and hit time, and compiler/data-layout effects.

Chapter 4: Pipelining Coverage includes IF/ID/EX/MEM/WB behavior, speedup equations, CPI stall modeling, the principle that program behavior plus hazards produces stalls, structural and control hazards, delay and squash-slot ideas, BTB behavior with and without tags, return-address stacks, and direction prediction heuristics.

Chapter 5: Integrated Review and Cache Simulation Concepts Coverage includes homework-aligned synthesis, worked pipeline calculations, hierarchy semantics, prefetching concepts, and interpretation of simulator-style statistics and AAT formulas.

Chapter 6: ILP and Superscalar Architecture Coverage includes CPI-versus-IPC interpretation, historical superscalar context, fire/complete ordering models, scoreboarding and busy-bit scheduling, function-unit latency considerations, register renaming intuition, and speculation/retirement behavior in out-of-order-friendly pipelines.

Chapter 1

Introduction

1.1 What Computer Architecture Is

Computer architecture is the discipline of designing computing systems that execute real workloads efficiently under practical constraints. In this course context, architecture means deciding how a processor and memory system should be organized so that useful work is completed quickly, reliably, and within power and cost limits. It is important to separate this from neighboring fields. Architecture is not transistor fabrication, circuit layout, or process chemistry, even though those influence what architects can build.

Every architectural decision is a tradeoff among competing goals. A design that minimizes latency for one task may reduce throughput for a server-style workload. A design that aggressively improves performance may consume more energy or require expensive cooling. A design that is elegant on paper may be too costly in area or too difficult to verify at scale. For this reason, architecture is best treated as constrained optimization: choose structures and policies that best match the target application while respecting implementation limits.

1.2 Core Methods: Parallelism, Speculation, and Prediction

Most performance techniques in modern processors can be traced to three closely linked ideas: *parallelism*, *speculation*, and *prediction*. *Parallelism* increases performance by doing multiple useful operations at the same time. *Speculation* increases performance by doing work early, before the machine is fully certain that this work will be needed. *Prediction* estimates uncertain future behavior (for example branch direction or branch target) so speculative work is directed toward the most likely path. Prediction is not a third independent speedup method; it is a tool that makes speculation more selective and efficient.

1.2.1 Speculation Without Prediction

Speculation does not require a learned predictor. A processor can speculate by optimistic overlap: continue execution under the assumption that an operation will complete normally, and recover only if that assumption fails. This is useful because waiting for every operation to retire before moving on would underutilize the pipeline.

Example: write buffer A write buffer is a standard example. When the core executes a store, it can enqueue the write and continue with subsequent instructions before that store reaches the

lower memory hierarchy. If the store later encounters a fault condition, architectural recovery and exception handling restore correctness. The processor did not predict branch direction or data values in this case; it simply advanced optimistically to avoid stalling.

1.2.2 Speculation With Prediction

When prediction is added, speculation becomes more selective and usually more efficient. The processor estimates likely future behavior, executes along the estimated path, and commits the work only if the estimate is correct.

Example: branch prediction Branch prediction is the canonical case. If the predictor guesses that a branch is taken, instruction fetch and decode proceed down the predicted target path immediately. A correct guess keeps the front end busy and reduces control-hazard stalls. An incorrect guess forces the machine to squash the speculative instructions and restart on the correct path, which introduces a misprediction penalty. The basic tradeoff is straightforward: high prediction accuracy yields high performance; low accuracy can erase the benefit of speculation.

1.3 Parallelism by Grain Size

Parallelism is best understood by *grain size*, meaning the amount of work grouped into one concurrent unit. At the smallest scale, bit-level parallelism appears in wide datapaths and arithmetic logic that operate on many bit positions at once. A 64-bit adder, for example, processes far more state per cycle than an 8-bit design because the hardware width itself exposes more fine-grained parallel work.

At *instruction-level parallelism (ILP)*, the machine overlaps independent instructions within a thread. Superscalar issue, out-of-order scheduling, and pipelining all exploit ILP. Consider two arithmetic instructions: `R1=R2+R3` and `R4=R5+R6`. Because neither depends on the other, a dual-issue machine can dispatch them together. In contrast, `R1=R2+R3` followed by `R4=R1+R6` introduces a data dependence that limits overlap. This illustrates the central ILP limitation: dependencies, control hazards, and hardware resource conflicts bound achievable parallel speedup.

At *thread-level parallelism (TLP)*, multiple threads execute concurrently on multithreaded or multicore hardware. TLP can provide substantial throughput gains, but only if the workload contains enough independent work and synchronization overhead remains controlled. Inter-program parallelism is the coarsest common level, where independent processes run simultaneously under an operating system scheduler. This is the dominant form in desktop multitasking and cloud services, where many programs share the same physical platform.

1.4 Flynn's Taxonomy

Flynn's taxonomy classifies parallel machines by counting instruction streams and data streams. In *SISD* (single instruction, single data), one instruction stream acts on one data stream, matching the traditional scalar uniprocessor model. In *SIMD* (single instruction, multiple data), one instruction stream is applied across many data elements in lockstep, which is highly effective for dense data-parallel computations such as vector math, image processing, and many GPU kernels.

In *MIMD* (multiple instruction, multiple data), multiple processing elements execute different instruction streams on different data streams. This is the dominant model for modern multicore

CPUs, distributed servers, and cluster systems because it supports general-purpose, irregular workloads. *MISD* (multiple instruction, single data) exists mostly in specialized contexts, such as certain fault-tolerant designs, and is uncommon in mainstream systems.

1.4.1 SIMD vs. MIMD

SIMD and MIMD solve different problems. SIMD is efficient when the same operation must be applied repeatedly to large collections of data. Its lockstep execution naturally provides synchronization across lanes, which reduces control overhead. MIMD is more flexible because each processor can follow its own control path, but this flexibility introduces synchronization and communication costs that must be managed explicitly by hardware and software.

Modern GPUs often use a hybrid model sometimes called *SIMT* (Single Instruction, Multiple Threads). SIMT provides a SIMD-like execution efficiency for the hardware while allowing a MIMD-like programming model where each thread appears to have its own scalar program counter. The hardware manages the divergence and convergence of these threads, grouping those on the same path into SIMD-style execution units.

1.4.2 Where pipelining fits

Pipelining is orthogonal to Flynn's taxonomy. It is a temporal form of parallelism in which different stages operate on different instructions concurrently. Because pipeline structure is an internal execution organization, pipelining can appear inside SISD cores, SIMD engines, and MIMD processors alike.

1.5 Vectors: Packed-Word SIMD vs. Pipelined Vector Units

Packed-word SIMD and classic vector pipelines both exploit data parallelism, but they do so with different execution structures. Packed-word SIMD divides a fixed-width register into lanes (for example, four 32-bit lanes in 128 bits) and applies one instruction across all lanes in lockstep. An operation such as `ADD_PACKED` performs multiple elementwise additions per instruction, and lane boundaries isolate carries so one lane's overflow does not spill into neighboring lanes. This style underlies CPU SIMD extensions such as SSE, AVX, and NEON.

Vector pipelines, in contrast, stream long vectors through deep arithmetic pipelines. A floating-point multiply can be viewed conceptually as operand preparation, multiply, and result normalization/writeback stages. If a pipeline has k stages and processes n elements, a serial implementation needs nk cycles, while a pipelined implementation needs approximately $k + n - 1$ cycles after fill effects are accounted for (Eq. (1.1)):

$$\text{serial cycles} = nk, \qquad \text{pipeline cycles} \approx k + n - 1. \tag{1.1}$$

The result is substantial throughput gain for long vectors even though single-element latency is not eliminated.

Chaining Vector chaining improves this further. If a second vector operation consumes results from a first operation, the hardware can forward early produced elements directly into the consumer pipeline instead of waiting for the full vector to complete. Chaining reduces producer-consumer delay and improves sustained throughput in vectorized loops.

1.6 Common Chapter 1 Pitfalls

Students often conflate speculation with prediction. The correct view is that prediction is one way to make speculation more effective, not the definition of speculation itself. Another recurring error is to assume that additional cores guarantee proportional speedup; real speedup is limited by dependencies, communication, and serial regions of code. It is also common to treat SIMD and MIMD as interchangeable, even though they target different workload structures and have different programming implications.

A final source of confusion is the difference between packed-word SIMD and long-vector pipelining. Both process multiple data elements, but packed SIMD exploits lane-level parallel arithmetic within instruction-width registers, while vector pipelines exploit staged streaming over long element sequences. Related to this, pipelining primarily improves throughput rather than the latency of a single operation.

1.7 Chapter 1 Summary

Chapter 1 introduces computer architecture as disciplined performance engineering under practical constraints. The chapter frames nearly all speedup mechanisms as forms of parallelism or speculation, then organizes parallelism by grain size from bits through instructions, threads, and entire programs. Flynn's taxonomy provides a clean language for distinguishing SISD, SIMD, MISD, and MIMD machines, especially the practical differences between SIMD efficiency and MIMD flexibility. Finally, the chapter distinguishes packed-word SIMD from pipelined vector execution and shows why chaining is valuable in vector data paths. Together, these ideas form the conceptual base used throughout later note volumes.

Chapter 2

Performance Evaluation

2.1 Why Performance Evaluation Matters

Architecture decisions are only meaningful if they are evaluated with the right metric. A design that improves one metric can degrade another, and a design that looks fast on one workload can be slow on the workload that actually matters. The central objective of Chapter 2 is therefore methodological: define performance correctly, choose representative workloads, and use mathematically correct summaries when comparing alternatives.

2.2 Throughput vs. Runtime

Two primary performance metrics appear repeatedly in computer architecture. *Runtime* (also called response time or turnaround time) measures how long one job takes from start to completion. *Throughput* measures how many jobs complete per unit time. These metrics are related but not interchangeable.

For a single-user interactive application, runtime is usually the dominant concern: the user wants one operation to finish quickly. For servers and shared systems, throughput is often dominant: the owner cares about how many requests the system can process per second. The same machine can be better under one metric and worse under the other, so performance claims must always specify which metric is being optimized.

This distinction also explains why pipelining can be valuable even when individual operation latency does not fall dramatically. A deeper pipeline can increase the number of completed operations per unit time, improving throughput, while leaving single-operation latency unchanged or slightly worse.

2.3 CPU Runtime Equation

The fundamental equation used throughout performance analysis is (Eq. (2.1)):

$$\text{CPU time} = \text{Instruction Count (IC)} \times \text{Cycles per Instruction (CPI)} \times \text{Cycle Time (CT)}. \tag{2.1}$$

Using clock frequency $f = 1/\text{CT}$, the same equation is (Eq. (2.2)):

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}}{f}. \tag{2.2}$$

This decomposition is useful because each factor is controlled by different mechanisms. *Instruction count* is influenced by algorithm choice, compiler quality, and ISA expressiveness. *Cycles per instruction (CPI)* is influenced by microarchitecture, hazard behavior, memory hierarchy behavior, and available parallelism. *Cycle time* is influenced by circuit and pipeline timing constraints.

For instruction mixes, total cycles are computed directly (Eq. (2.3)):

$$\text{Total cycles} = \sum_i (\text{IC}_i \times \text{CPI}_i), \qquad \text{CPI}_{avg} = \frac{\text{Total cycles}}{\text{Total instructions}}. \tag{2.3}$$

Worked example Suppose a workload executes 76 loads at 6 cycles each, 30 stores at 4 cycles each, 300 adds at 1 cycle each, and 58 multiplies at 5 cycles each. Then total cycles are

$$76 \cdot 6 + 30 \cdot 4 + 300 \cdot 1 + 58 \cdot 5 = 1166. \tag{2.4}$$

At 2.2 MHz, runtime is

$$\frac{1166}{2.2 \times 10^6} \approx 5.30 \times 10^{-4}\text{s} = 0.53\text{ms}. \tag{2.5}$$

If multiply latency is reduced from 5 cycles to 3 cycles, new total cycles become

$$76 \cdot 6 + 30 \cdot 4 + 300 \cdot 1 + 58 \cdot 3 = 1050, \tag{2.6}$$

which yields a speedup of

$$\frac{1166}{1050} \approx 1.11. \tag{2.7}$$

The example illustrates a recurring theme: accelerating only one instruction class can help, but total improvement depends on how often that class occurs.

2.4 RISC vs. CISC Through the Performance Equation

RISC and *CISC* are best understood as design tradeoffs, not as a simple "better/worse" ranking. Historically, CISC designs emphasized richer, denser instructions to reduce dynamic instruction count and code size. RISC designs emphasized simpler instructions and more regular pipelines, often improving CPI and enabling higher clock rates. In CPU-time terms, this is a direct trade (Eq. (2.8)):

$$\text{IC} \downarrow \text{may come with} \text{CPI} \uparrow \text{or} \text{CT} \uparrow, \quad \text{or vice versa}. \tag{2.8}$$

In modern systems the boundary is less rigid than textbook definitions suggest, and x86 implementations, for example, decode complex instructions into simpler internal micro-operations and then execute them with highly RISC-like back ends. ARM designs, traditionally associated with RISC, also include increasingly rich instruction features to improve efficiency for modern workloads. The practical conclusion is that ISA style should be evaluated by end-to-end performance, power, and software ecosystem impact rather than by labels alone.

2.5 Amdahl's Law

Amdahl's Law quantifies the speedup limit when only part of a workload is accelerated. If fraction f of execution time benefits from a speedup factor s, total speedup is

$$S_{Amdahl} = \frac{1}{(1-f) + \frac{f}{s}}. \tag{2.9}$$

The limit as $s \to \infty$ is

$$S_{\text{upper}} = \frac{1}{1-f}. \tag{2.10}$$

This bound is the key insight: the unaccelerated fraction eventually dominates.

Worked example If 80% of runtime is parallelizable and the accelerated part can use 100,000 processors ideally ($s = 100{,}000$), then

$$S = \frac{1}{0.2 + \frac{0.8}{100000}} \approx 5.0. \tag{2.11}$$

Even enormous parallel resources cannot overcome a significant serial fraction. This is why architectural proposals must report both the accelerated fraction and the achieved acceleration factor.

2.6 Gustafson's Law and Why It Does Not Contradict Amdahl

Gustafson's Law addresses a different question: instead of fixing problem size, it assumes the workload scales with available processors. Let α be the sequential fraction measured on an N-processor run. Scaled speedup is

$$S_{Gustafson} = \alpha + (1-\alpha)N. \tag{2.12}$$

This often grows nearly linearly with N when α is small.

Amdahl and Gustafson do not contradict each other because their assumptions differ. Amdahl is *strong-scaling* analysis (fixed total work). Gustafson is *weak-scaling* analysis (larger total work as resources grow). Both are valid, but they answer different engineering questions.

Example With $\alpha = 0.15$ and $N = 16$,

$$S_{Gustafson} = 0.15 + 0.85 \cdot 16 = 13.75. \tag{2.13}$$

This can exceed Amdahl's fixed-size bound because the scaled workload includes more total useful work.

2.7 Combining Performance Numbers Correctly

Single-number summaries are useful only when the averaging method matches the quantity being averaged. For times (seconds per task), *arithmetic mean* is appropriate (Eq. (2.14)):

$$\bar{T}_{arith} = \frac{1}{n}\sum_{i=1}^{n} T_i. \tag{2.14}$$

For rates (tasks per second), use *harmonic mean* (Eq. (2.15)):

$$\bar{R}_{harm} = \frac{n}{\sum_{i=1}^{n} \frac{1}{R_i}}. \tag{2.15}$$

Using arithmetic mean on rates can select the wrong system. Suppose machine A runs two workloads at 4 and 4 transactions/s, while machine B runs them at 2 and 7 transactions/s. Arithmetic means are 4.0 and 4.5, which appears to favor B. But for one transaction of each workload, total throughput is determined by total work divided by total time (Eq. (2.16)):

$$R_A = \frac{2}{\frac{1}{4}+\frac{1}{4}} = 4, \qquad R_B = \frac{2}{\frac{1}{2}+\frac{1}{7}} = \frac{28}{9} \approx 3.11. \tag{2.16}$$

Under equal-work comparison, A is better. This is exactly why rates should be combined harmonically, not arithmetically.

2.8 Workload Dependence and Benchmarking

Performance numbers are workload-dependent. Miss ratio, CPI, and even effective clock behavior under thermal constraints depend on the executed program mix. A claim such as "processor X is faster" is incomplete unless it states the benchmark set, input sizes, and aggregation method.

Representative benchmarking therefore requires matching the measured workloads to expected deployment behavior. A web service, a game engine, and a scientific simulation stress very different execution paths and memory behaviors. Chapter 2 methodology demands that architects evaluate on relevant traces and report assumptions explicitly; otherwise conclusions will not transfer to real systems.

2.9 Chapter 2 Summary

Chapter 2 provides the framework for disciplined performance reasoning. Runtime and throughput are distinct objectives and must not be mixed casually. CPU time decomposes into IC, CPI, and cycle time, which allows precise reasoning about optimization impact. RISC and CISC are best evaluated as tradeoff points in Eq. (2.1) rather than as absolute categories. Amdahl's Law gives hard limits for fixed-size speedup, while Gustafson's Law explains scaled-workload growth without contradiction. Finally, performance summaries require correct statistics: arithmetic means for time, harmonic means for rate. These tools form the quantitative core used in later chapters.

Chapter 3

Cache Memories

3.1 Memory Technology and the Hierarchy

Modern cache design begins with a technology reality: fast memory is expensive and dense memory is slower. *SRAM* cells use more transistors per bit, provide lower access latency, and do not require refresh, so SRAM is the natural choice for on-chip caches. *DRAM* cells are denser and cheaper per bit but slower and refresh-driven, so DRAM is the natural choice for main memory capacity. This is why systems are built as hierarchies rather than as one uniform memory.

A useful mental model is that each level trades speed for size as distance from the core increases. L1 and often L2 are SRAM-based and optimized for very low hit latency. Lower levels and main memory prioritize capacity and cost. Architecture work in Chapter 3 is mostly about exploiting this hierarchy so that accesses are absorbed by fast levels as often as possible.

At the circuit level, one practical limiter is wire delay. Longer bit lines have higher effective capacitance and resistance, so sense amplifiers need more time to resolve values. This is one reason very large single-level caches become slow: making arrays taller increases bit-line delay even if logic depth is unchanged.

3.2 Typical Hierarchy Levels

In mainstream systems, the first level is a small L1 cache optimized for minimum hit time. A larger L2 cache is usually next, trading some latency for much lower miss ratio. Below L2 sits physical memory (DRAM), which is much larger but much slower. Beneath DRAM is virtual-memory backing store (for example SSD), managed by the operating system through paging.

From the CPU perspective, each lower level is accessed only when the upper level misses. This staged access is exactly why average-access-time analysis composes recursively across levels.

3.3 Locality: Why Caches Work

Cache behavior is dominated by *locality*. *Temporal locality* means that if data is used now, it is likely to be used again soon. *Spatial locality* means that if one address is accessed, nearby addresses are likely to follow. *Sequential locality* is a common special case of spatial locality in which accesses proceed in order, such as instruction fetch or array traversal.

These properties justify block-based transfer. Instead of moving one byte at a time from DRAM, the system moves an entire cache block, betting that adjacent bytes will be useful soon. The same properties also justify replacement policies that retain recently used data and evict older candidates.

3.4 Cache Geometry and Address Partitioning

To reason cleanly about cache structure, use the triplet (C, B, S) (Eq. (3.1)):

$$\text{cache capacity} = 2^C \text{ bytes}, \qquad \text{block size} = 2^B \text{ bytes}, \qquad \text{associativity} = 2^S \text{ ways}. \tag{3.1}$$

Address fields in (C, B, S) notation In this notation, S is **not** an extra address field beyond tag/index/offset. Instead, S changes the number of sets and therefore the index width.

$$\text{sets} = \frac{2^C}{2^B \cdot 2^S} = 2^{C-B-S}. \tag{3.2}$$

Total line count follows directly from sets and ways (Eq. (3.3)):

$$\text{lines} = \text{sets} \cdot \text{ways} = 2^{C-B-S} \cdot 2^S = 2^{C-B}. \tag{3.3}$$

For direct-mapped caches ($S = 0$), ways $= 1$ so lines $=$ sets. For set-associative caches ($S > 0$), lines $>$ sets because there are multiple ways per set.

$$|\text{Offset}| = B, \qquad |\text{Index}| = \log_2(\text{sets}) = C - B - S. \tag{3.4}$$

Because index plus offset consume $(C - S)$ low-order bits (Eq. (3.5)):

$$|\text{Index}| + |\text{Offset}| = (C - B - S) + B = C - S. \tag{3.5}$$

For an address width of AW bits (Eq. (3.6)):

$$|\text{Tag}| = AW - (|\text{Index}| + |\text{Offset}|) = AW - (C - S). \tag{3.6}$$

So the split remains (Eq. (3.7)):

$$\text{Address} = [\text{Tag} \mid \text{Index} \mid \text{Offset}], \tag{3.7}$$

with

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = AW - (C - S). \tag{3.8}$$

Where the S bits "go" Higher associativity means fewer sets, so index width decreases by S relative to a direct-mapped organization with the same C and B. That is why S appears in index/tag equations but is not carried as an address field.

What selects the way? The way is selected by hardware after tag comparison, not by an explicit address field:

1. Index selects one set.
2. Tags for all 2^S ways in that set are compared in parallel.
3. Match logic identifies the hitting way (if any).
4. Internal way-select control drives the data mux (conceptually an S-bit code since $\log_2(2^S) = S$).

Quick example: 32 KB, 32 B blocks, 4-way With $(C, B, S) = (15, 5, 2)$ (Eq. (3.9)):

$$|\text{Index}| = C - B - S = 15 - 5 - 2 = 8. \tag{3.9}$$

For the same C, B but direct-mapped ($S = 0$) (Eq. (3.10)):

$$|\text{Index}| = 15 - 5 - 0 = 10. \tag{3.10}$$

So moving to 4-way associativity reduces index width by 2 bits (equivalently reduces sets by 4×).

32-bit address specialization If $AW = 32$ (Eq. (3.11)):

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = 32 - (C - S) = 32 - C + S. \tag{3.11}$$

Worked example (metadata sizing) Consider a 64-bit machine with $(C, B, S) = (13, 5, 0)$, an 8 KB direct-mapped cache with 32-byte blocks. The cache has $2^{13-5} = 256$ blocks and $2^{13-5-0} = 256$ sets. Tag width is $64 - (13 - 0) = 51$ bits. If each block stores one valid bit and one dirty bit, metadata per block is 53 bits. Total tag-store plus metadata is (Eq. (3.12)):

$$256 \times 53 = 13{,}568 \text{ bits}. \tag{3.12}$$

If B is reduced to 3 while C and S are fixed, the number of blocks rises to $2^{10} = 1024$, so tag overhead increases to $1024 \times 53 = 54{,}272$ bits. This illustrates an important Chapter 3 point: smaller blocks can increase tag overhead significantly.

Address partitioning example Consider the same $(13, 5, 0)$ 64-bit cache. For the virtual address `0000_1234_ABCD_5678`: **Offset (5 bits):** the lowest 5 bits of `78` (`0111_1000`) are `11000`, which is `18` (decimal 24). **Index (8 bits):** bits 5 through 12 come from hex digits `5`, `6`, and `7`. In binary these digits are `0101`, `0110`, and `0111`; extracting bits 5–12 yields `1011_0011` (binary), which is `B3`. **Tag (51 bits):** the remaining upper bits are the tag. This direct bit-mapping is the fundamental operation performed by hardware on every memory access. It illustrates how physical cache structure determines exactly which address bits are used for indexing and comparison.

3.5 Direct-Mapped, Set-Associative, and Fully Associative

A *direct-mapped cache* has one way per set, so each block address maps to exactly one location. Lookup is simple and fast, but conflict misses can be severe. A *set-associative cache* allows each index to hold multiple candidate blocks; it reduces conflicts at the cost of parallel tag comparisons and a way-select multiplexer. A *fully associative cache* removes index constraints entirely and minimizes conflict misses, but hardware search complexity and power become substantial for large capacities.

Under (C, B, S) notation, direct-mapped means $S = 0$. Fully associative means one set, so $C - B - S = 0$ and therefore $S = C - B$. This distinction is critical, because confusing these two conditions leads to incorrect address math and wrong simulator behavior.

How each cache is constructed Direct-mapped construction uses one data line and one tag entry per index. The index decoder selects one row, then one tag compare and one word select complete the access. A 2^S-way set-associative cache replicates tag and data entries 2^S times per index; all tags in the selected set are compared in parallel, and a way-select mux picks the winning line. A fully associative cache has a single set containing all lines, so every access performs an associative search across the entire tag store and relies heavily on replacement metadata.

3.6 Hit, Miss, and Average Access Time

For a cache hit, access latency is the *hit time* HT. For a miss, the processor first spends hit-time logic discovering the miss, then pays *miss penalty* MP to fetch from the next level. The *average access time* is (Eq. (3.13)):

$$AAT = HT + MR \cdot MP, \tag{3.13}$$

where MR is *miss ratio*. In Chapter 3 analysis, Eq. (3.13) is the cache analogue of the CPU-time equation in Eq. (2.1).

Hit time is not a single gate delay. It includes row decoding, word-line drive, bit-line sensing, tag comparison, way selection, and word/byte selection. Cache-geometry choices alter these terms. Taller arrays hurt bit-line delay, wider arrays increase mux and comparator complexity, and higher associativity increases tag-compare and selection cost.

3.7 The Three C's of Misses

Misses are classed as *compulsory*, *capacity*, or *conflict*. Compulsory misses are first-touch misses and are unavoidable unless data is brought in before first demand access. Capacity misses happen because the working set exceeds what the cache can hold over the reuse interval. Conflict misses happen when mapping restrictions force evictions even though total capacity would have been enough under freer placement.

In practice, this taxonomy provides a diagnosis strategy. If misses are mainly conflict-driven, associativity or victim structures can help. If misses are mainly capacity-driven, larger effective storage or better blocking can help. If misses are mainly compulsory, prefetching and layout changes are primary tools.

3.8 Replacement Policies and Their Tradeoffs

When a set is full and a miss occurs, one resident block must be evicted. *Belady's optimal policy* is a theoretical lower bound that requires future knowledge and is not implementable directly. *LRU* approximates this by evicting the least recently touched entry, and usually performs well under temporal locality. *FIFO* ignores reuse and can evict hot lines. *Random* is simple and sometimes surprisingly competitive but unpredictable. *LFU* tracks frequency but can lag phase changes.

For high associativity, exact LRU state grows costly. Practical designs use approximations, tree-based encodings, or hybrid insertion policies. A notable policy distinction is insertion behavior: a line can be installed as *MRU* (default in many demand-miss paths) or near LRU (useful for speculative prefetch lines to reduce pollution).

3.9 Write Policies

Write behavior has two independent decisions. On a write hit, either update only the cache and defer memory update (*write-back*), or update both cache and memory immediately (*write-through*). On a write miss, either fetch the block then write it (*write-allocate*), or write to lower memory without installing in cache (*write-no-allocate*).

Write-back reduces memory traffic when repeated writes target the same line, but requires a dirty bit and careful eviction handling. Because dirty data can live in cache before eviction, abrupt power loss can drop recently written values that have not yet reached lower levels. Write-through simplifies

coherence visibility and durability behavior because lower memory is updated on each write hit, and it does not require dirty metadata at that cache level, but it increases memory bandwidth pressure. The common pairings are write-back plus write-allocate for reuse-heavy data paths, and write-through plus write-no-allocate for streaming paths with little read-after-write reuse.

Valid and dirty bits: meaning, need, and size impact Each cache line typically carries at least one *valid bit*. Valid= 1 means the tag/data entry contains a meaningful memory block; valid= 0 means the entry must not hit, even if tag bits happen to match after reset or replacement. A *dirty bit* is needed when that cache level uses *write-back*: dirty= 1 means the line was modified in cache and must be written to the next level before eviction; dirty= 0 means eviction can discard the line without write-back traffic. In a pure write-through level, dirty bits are not required because lower memory is updated on every write hit.

If line count is $N_{lines} = 2^{C-B}$, valid-bit storage is N_{lines} bits. Dirty-bit storage is another N_{lines} bits only when write-back is enabled at that level. Relative to data-array size 2^C bytes, overhead from these two bits alone is (Eq. (3.14)):

$$\text{overhead fraction} = \frac{n_v + n_d}{2^{B+3}}, \tag{3.14}$$

where $n_v = 1$ (valid always) and $n_d \in \{0, 1\}$ (dirty only for write-back). Example: with 32-byte blocks ($B = 5$), valid-only overhead is $1/256 \approx 0.39\%$, and valid+dirty overhead is $2/256 \approx 0.78\%$ (excluding tag, replacement, and coherence bits).

3.10 Reducing Miss Ratio

The first lever is **cache size (C)**. Increasing C reduces capacity misses because more of the working set fits in the hierarchy. However, larger caches increase hit time due to longer bit lines and decoder delays, and they consume more area and power.

The second lever is **associativity (S)**. Increasing S reduces conflict misses by providing more locations for a given index. The benefit is most significant when moving from direct-mapped to 2-way or 4-way, after which returns diminish. The cost includes increased hit time (due to the way-select multiplexer and parallel tag comparisons) and increased power consumption.

The third lever is **block size (B)**. Increasing B reduces compulsory misses by exploiting spatial locality (fetching more useful data per miss). However, if B becomes too large, it can increase conflict misses (by reducing the total number of blocks) and will always increase miss penalty (longer transfer time).

Hardware Prefetching Hardware prefetching targets compulsory and capacity effects by pulling data before demand. A +1 *prefetcher* fetches block $i+1$ when block i is accessed. A *stride prefetcher* detects constant deltas (for example $i, i+k, i+2k$) and fetches the next predicted address. A *Markov prefetcher* uses a correlation table to learn non-linear transitions; if address A is often followed by B then C, the predictor learns and issues those future blocks when A appears. To prevent *cache pollution* (evicting useful data for speculative data that might not be used), many designs use a *prefetch buffer*. This is a small, separate storage area where prefetched lines sit until they are actually demanded by the CPU. If demanded, they are moved into the main cache.

3.11 Reducing Miss Penalty

Miss penalty is shaped strongly by transfer mechanics. If a block has 2^B bytes and bus transfer width is W bytes, then the number of sub-block transfers is $N_{sb} = 2^B/W$. With first-transfer startup latency T_1 and per-following transfer time T_f, block transfer time is (Eq. (3.15)):

$$MP_{full} = T_1 + (N_{sb} - 1)T_f. \tag{3.15}$$

Early restart returns data to the CPU as soon as the requested sub-block arrives rather than after full-block completion. If critical-word position is uniformly distributed and transfer order is fixed from sub-block 0 upward, average CPU-visible penalty becomes (Eq. (3.16)):

$$MP_{ER} \approx T_1 + \frac{N_{sb} - 1}{2} T_f. \tag{3.16}$$

Critical-word-first improves further by requesting the demanded sub-block first, then wrapping to complete the rest in the background.

Another major lever is adding lower cache levels. If L1 misses probe L2 before DRAM, L1 penalty becomes L2 average access time (Eq. (3.17)):

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot AAT_{L2}. \tag{3.17}$$

This recursive form extends naturally to deeper hierarchies.

Victim caches are also miss-penalty tools. A small fully associative buffer captures lines evicted from the primary cache. On a later conflict-driven re-reference, data can be swapped back quickly without going to DRAM. Conceptually, a victim cache provides a small amount of flexible associativity exactly where pressure appears.

Write buffers reduce stalls by decoupling cache writes from lower-memory completion. They are especially useful when a following miss would otherwise wait behind pending writes. Correct implementation requires forwarding from the newest matching buffered write and careful ordering semantics.

3.12 Reducing Hit Time

Hit-time optimization frequently conflicts with miss-rate optimization. Smaller arrays and lower associativity usually improve hit latency. However, overly aggressive simplification can raise miss ratio enough to hurt total performance. In general, small caches and direct-mapped organizations are fast on the hit path, but they are often unacceptable unless miss ratio remains controlled by workload behavior or lower-level caches.

Address translation complicates this tradeoff. A naive *physically indexed, physically tagged* path can serialize translation lookaside buffer (TLB) lookup before cache access, increasing L1 hit latency. *Virtually indexed, physically tagged (VIPT)* caches overlap TLB translation with set indexing and preserve physical-tag correctness.

PIPT vs. VIPT timing view With virtual memory, the CPU generates a virtual address split into VPN and page offset. In a *physically indexed, physically tagged (PIPT)* L1, both index and tag use physical address bits. That means the TLB must produce the physical address before cache indexing can begin. So a first-order timing model is (Eq. (3.18)):

$$HT_{L1,\text{PIPT}} \approx T_{TLB} + T_{L1,\text{index+array+tag}}. \tag{3.18}$$

In a *VIPT* L1, indexing starts immediately using low-order virtual bits while the TLB is translating in parallel. When the physical address arrives, the physical tag compare finalizes hit/miss. A first-order model is (Eq. (3.19)):

$$HT_{L1,\text{VIPT}} \approx \max(T_{TLB},\, T_{L1,\text{index+array}}) + T_{\text{phys_tag_compare}}. \tag{3.19}$$

The practical effect is that VIPT can hide much of TLB latency on the L1 hit path, while PIPT often exposes it directly.

Why the VIPT constraint exists For VIPT to work cleanly, index-plus-offset bits must fit within page-offset bits (Eq. (3.20)):

$$C - S \leq P, \tag{3.20}$$

where P is page-offset width in bits. If $C-S > P$, then some index bits come from VPN bits. Different virtual addresses that map to the same physical page (*synonyms*) can then index different sets, allowing duplicated physical data in L1. That creates alias-management problems: writes through one synonym may not immediately update the other cached copy, risking stale reads until extra coherence/synchronization machinery repairs it. This constraint often forces minimum associativity for a given L1 size and page size. For example, with $P = 12$ (4 KB pages) and $C = 15$ (32 KB L1), one needs $S \geq 3$, i.e., at least 8-way associativity. If $S = 2$ (4-way), then $C - S = 13 > 12$, so VIPT indexing would extend beyond page offset and synonym risk appears.

Write buffers can also improve effective hit behavior for writes by decoupling write completion from the immediate critical path and by forwarding recent write data when needed. This prevents some write-after-read and read-after-write timing stalls from stretching apparent hit latency.

3.13 Virtual Memory Technology, Terminology, and Analogy to Caching

Virtual-memory terminology mirrors cache terminology closely. A virtual address is typically split into a *virtual page number (VPN)* and page offset. Translation maps VPN to a *physical frame number (PFN)* through a page table. A *translation lookaside buffer (TLB)* is a cache of recent translations. A missing translation in the TLB causes a page-table walk; if the page is not in DRAM, a page fault triggers OS handling and secondary-storage transfer.

The analogy to caching is direct. VPN behaves like a tag, page offset behaves like block offset, and the TLB behaves like a small associative cache of mapping metadata. Likewise, DRAM behaves as a cache for backing store, while on-chip caches behave as caches for DRAM. This layered view is useful because it applies one conceptual framework to both hardware cache hierarchy and OS-managed virtual memory.

3.14 Compiler and Data-Layout Techniques

Software organization strongly affects cache performance. **Compiler layout of instructions** places frequently executed basic blocks close together in memory to improve I-cache hit rate and reduce fetch stalls. **Compiler-directed prefetching** issues non-binding prefetch instructions ahead of use, which is effective for predictable array accesses. **Data-layout choices**, including structure-of-arrays (SoA), field reordering, and padding, can reduce cache-line waste and false sharing.

Tiling (blocking) re-organizes loops to process data in smaller cache-resident regions. For example:

```
// Before tiling:
for (i=0; i<N; i++) for (j=0; j<N; j++) A[i][j] = B[j][i];
// After tiling (where b is the tile size):
for (ii=0; ii<N; ii+=b) for (jj=0; jj<N; jj+=b)
    for (i=ii; i<ii+b; i++) for (j=jj; j<jj+b; j++) A[i][j] = B[j][i];\label{verb:verbatim}\labe
```

Loop fusion combines independent loops that touch related data to improve temporal locality. For example:

```
for (i=0; i<N; i++) a[i] = b[i] + 1; // loop 1
for (i=0; i<N; i++) c[i] = a[i] * 2; // loop 2
// Fused:
for (i=0; i<N; i++) { a[i] = b[i] + 1; c[i] = a[i] * 2; }\label{verb:verbatim2}\label{verb:verba
```

In practice, these compiler and data-layout techniques frequently determine whether hardware cache features help or saturate.

3.15 Chapter 3 Summary

Chapter 3 develops cache memory from first principles to practical optimization. The chapter links SRAM/DRAM technology limits to hierarchical design, formalizes cache geometry with (C, B, S) math, and uses AAT to connect structure to performance. Misses are interpreted through the three C's, while replacement and write policies define behavioral tradeoffs. Improvement strategies are organized by target term: miss ratio, miss penalty, or hit time. Prefetching, victim caches, write buffers, multi-level hierarchies, and VIPT constraints show how modern designs balance speed, capacity, and correctness. Finally, compiler and data-layout techniques complete the picture by showing that cache performance is a hardware-software co-design problem.

Chapter 4

Pipelining

4.1 Pipeline Motivation

Pipelining improves throughput by overlapping instruction execution stages in time. Instead of waiting for one instruction to complete all work before starting the next, the processor places stage registers between major datapath functions and allows multiple instructions to be in flight simultaneously. The key gain is not that one instruction becomes instantly shorter in latency, but that completed instructions per unit time rise after the pipeline fills.

4.2 Five-Stage Example Pipeline

The canonical integer pipeline has five stages (Eq. (4.1)):

$$IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB. \tag{4.1}$$

Instruction Fetch (IF) reads the instruction from the instruction memory hierarchy and advances the PC. Instruction Decode (ID) decodes opcode and operands and reads source registers. Execute (EX) performs ALU work and effective-address calculation. Memory (MEM) performs data-memory access for loads and stores. Write Back (WB) writes results to the architectural register file.

Even instructions that do not logically need memory often still occupy the MEM stage in a basic in-order five-stage design so that all instructions remain aligned in a uniform stage progression. This simplifies control and preserves in-order retirement semantics.

4.3 Pipeline Speedup Calculation

Suppose an unpipelined design executes one instruction in N stage-equivalent delays, each of delay T_s. Its instruction time is approximately NT_s. A pipelined design with stage registers has clock (Eq. (4.2)):

$$T_{clk,pipe} \approx T_s + T_{reg}, \tag{4.2}$$

where T_{reg} is latch/flip-flop overhead. For a long stream, ideal CPI approaches 1, so time per instruction approaches $T_{clk,pipe}$ and ideal speedup becomes (Eq. (4.3)):

$$S_{ideal} \approx \frac{NT_s}{T_s + T_{reg}}. \tag{4.3}$$

If T_{reg} were zero, this approaches N. In practice, register overhead and stalls reduce realized speedup.

Finite-length form For M instructions in an N-stage pipeline (ignoring hazards), total cycles are $N + M - 1$, so (Eq. (4.4)):

$$T_{pipe} \approx (N + M - 1)T_{clk,pipe}, \qquad T_{unpiped} \approx MNT_s. \tag{4.4}$$

This shows fill and drain overhead is more visible for short instruction sequences.

4.4 CPI Model and Stall Accounting

In steady state without hazards, ideal CPI is 1 (Eq. (4.5)):

$$CPI_{ideal} = 1. \tag{4.5}$$

With hazards, the practical model is (Eq. (4.6)):

$$CPI = 1 + \text{stalls}_{avg}, \tag{4.6}$$

where stalls_{avg} is average stall cycles per instruction. This form is central in Chapter 4: almost every pipeline optimization is either reducing stall frequency, reducing stall duration, or both.

4.5 Program Characteristics Plus Hazards Produce Stalls

Hardware hazards are potential conflicts. A stall occurs only when the running program triggers that conflict. This yields a guiding principle (Eq. (4.7)):

$$\text{Program characteristics} + \text{pipeline hazards} \Rightarrow \text{stalls}. \tag{4.7}$$

For example, a branch hazard matters only when branches occur in the dynamic instruction stream. A structural hazard from a shared memory port matters only when fetch and data access overlap in a conflicting cycle.

4.6 Structural Hazards

Structural hazards arise from insufficient hardware resources. A classic example is a single-ported memory system that must serve both IF and MEM stages. If one instruction needs data memory while another needs instruction fetch in the same cycle, one must wait, causing a stall.

Common avoidance strategies are straightforward architectural provisioning: split L1 into separate instruction and data caches, add ports, duplicate critical units, or schedule around known bottlenecks. In modern designs, many obvious structural hazards are eliminated in hardware because their stall cost is predictable and severe.

4.7 Data Hazards

Data hazards occur when the pipeline changes the order of read/write accesses to operands so that the order differs from the order in the original sequential program.

RAW (Read After Write) A *RAW hazard* occurs when an instruction refers to a result that has not yet been calculated or stored by a preceding instruction. This is the most common hazard in the classic 5-stage pipeline.

```
I1:  ADD R1, R2, R3    I2:  SUB R4, R1, R5
```
(4.8)

In this case, `I2` needs `R1` in its ID/EX stage, but `I1` does not write `R1` until its WB stage. Without intervention, `I2` would read a stale value.

WAR and WAW *WAR* (Write After Read) and *WAW* (Write After Write) hazards do not typically occur in the simple 5-stage in-order pipeline because instructions read early (ID) and write late (WB) in a fixed order. However, they become critical in out-of-order execution or pipelines with multiple completion units of varying latencies.

4.7.1 Forwarding (Bypassing)

Forwarding allows a result to be used as an input to a subsequent instruction before it is written to the register file. If the result is already available in a pipeline register (such as EX/MEM or MEM/WB), hardware can forward it directly to the ALU inputs. This eliminates many RAW stalls.

4.7.2 Load-Use Stalls

Forwarding cannot solve every RAW hazard. If a load instruction is followed immediately by an instruction that uses the loaded value, the data is not available until the end of the MEM stage, which is too late for the EX stage of the following instruction.

```
I1:  LW R1, 0(R2)    I2:  ADD R3, R1, R4
```
(4.9)

This case requires a one-cycle load-use stall even with forwarding. Compilers attempt to schedule independent instructions into these "load-delay" slots to maintain performance.

4.8 Control Hazards

Control hazards occur because the fetch stream does not immediately know the correct next PC for branches and jumps. The branch creates uncertainty in both target and direction (taken/not taken), and the pipeline may fetch wrong-path instructions that must later be squashed.

CPI impact example If branches occur every five instructions on average (branch frequency = 0.2) and each branch introduces one average stall cycle in a simple pipeline, then

$$CPI = 1 + 0.2 \times 1 = 1.2. \tag{4.10}$$

This simple calculation captures why even modest branch penalties have visible throughput cost.

4.8.1 Delay Slots

A *delay slot* ISA semantic says that one or more instructions after a branch execute before control transfer takes effect. The compiler tries to fill these slots with useful independent work. This can reduce explicit branch stalls when suitable instructions exist, but becomes increasingly difficult as pipelines deepen and dependence constraints tighten.

4.8.2 Squash Slots with Likely Bits

Some designs expose *squash-slot* behavior via branch-likely style semantics. The compiler places a candidate instruction near the branch and encodes likely behavior. Hardware keeps or squashes that instruction based on actual branch resolution and likely-bit policy. This improves performance when heuristics are good but complicates ISA semantics and compiler scheduling.

4.9 Branch Target Buffer (BTB)

The *BTB* predicts where a branch goes (target prediction) early enough to guide fetch.

4.9.1 BTB with Tags

A tagged BTB stores branch PC tags and associated targets. On fetch, the BTB is indexed and tag-compared; a valid tag hit yields a predicted target. Tagging eliminates *aliasing*, where two unrelated branches share an entry, ensuring higher prediction accuracy at the cost of significantly more hardware area and longer lookup time.

4.9.2 BTB without Tags

A tagless BTB behaves like a hashed direct lookup table from index to target metadata. It is fast and compact but prone to *aliasing*: unrelated branches may map to the same entry and pollute each other's predictions. This can still be acceptable if the table is sized well and collision rate is low for target workloads, trading accuracy for substantial area savings.

4.9.3 Hardware Return Address Stack

Function returns are indirect branches whose targets are difficult for generic BTB logic. A small hardware *return-address stack (RAS)* records call return PCs and pops on return, producing very high return-target accuracy in common call/return patterns. This is one of the most cost-effective branch-target optimizations.

4.10 Whether-To Branch: Direction Prediction

Target prediction answers where a branch goes. *Direction prediction* answers whether it is taken. A simple static method is software heuristics, especially *backward taken / forward not taken (BTFNT)*: loop back-edges are often taken, many forward conditionals are often not taken. Compilers can also use profile-guided statistics to set likely-direction metadata (for example branch-likely annotations).

Dynamic predictors improve further by learning runtime behavior (for example saturating counters and history-based schemes). The essential Chapter 4 lesson is that deeper pipelines require higher direction accuracy because misprediction penalty grows with the number of in-flight wrong-path stages.

4.11 Timing-Diagram Interpretation (Appendix Reference)

A full checklist for reading 5-stage timing diagrams is provided in Appendix A. That appendix covers structural, data, control, and memory cues, plus CPI and stall-attribution methods for worked traces.

4.12 Chapter 4 Summary

Chapter 4 frames pipelining as throughput engineering with explicit hazard management. The five-stage model clarifies stage roles (IF, ID, EX, MEM, WB), while speedup equations show why ideal gains are reduced by latch overhead and stalls. CPI is modeled as $1 + \text{stalls}_{avg}$, with stalls explained by the interaction of program behavior and hazard structure. Structural hazards are reduced through resource provisioning; control hazards are addressed through delay/squash semantics and prediction structures. BTBs predict branch targets, RAS handles returns efficiently, and direction prediction determines whether control transfer occurs, with software heuristics like backward taken/forward not taken providing a baseline. Together these ideas define practical pipeline performance analysis and design.

Chapter 5

Integrated Review and Cache Simulation Concepts

5.1 Chapter Purpose

This chapter integrates the concepts from Chapters 1–4 into a single combined overview.

5.2 Homework Themes from Chapters 1 and 2

Core concept clusters include classifying optimizations as parallelism, speculation, or both; distinguishing throughput from runtime and selecting the correct metric; interpreting CPU time = IC × CPI × CT as a tradeoff framework; applying Amdahl's law for fixed-size speedup limits; interpreting Gustafson's law for scaled-size behavior; and using arithmetic means for times versus harmonic means for rates.

Worked check If an optimization accelerates 30% of runtime by 2×, total speedup is

$$S = \frac{1}{(1-0.3)+\frac{0.3}{2}} = \frac{1}{0.85} \approx 1.176. \tag{5.1}$$

The unaccelerated fraction remains in the denominator and bounds overall speedup.

5.3 Homework Themes from Chapter 3

Core concept clusters include computing sets, ways, and tag/index/offset widths from (C, B, S); interpreting compulsory, conflict, and capacity misses; analyzing replacement and write-policy behavior; applying $AAT = HT + MR \cdot MP$; and relating cache size, block size, and associativity to hit time, miss ratio, and miss penalty.

Worked check For a cache with $HT = 1.8$ ns, $MR = 0.06$, and $MP = 28$ ns (Eq. (5.2)):

$$AAT = 1.8 + 0.06 \cdot 28 = 3.48 \text{ ns}. \tag{5.2}$$

5.4 Chapter 4 Practice Problems (with Answers)

Practice One: Pipeline Speedup

Assume a 5-stage design where each stage has 250 ps logic delay and pipeline-register overhead is 30 ps. Unpipelined instruction time is $5 \times 250 = 1250$ ps. Pipelined clock is $250 + 30 = 280$ ps. Ideal long-run speedup is (Eq. (5.3)):

$$S \approx \frac{1250}{280} \approx 4.46. \tag{5.3}$$

This is below 5 because register overhead is non-zero.

Practice Two: CPI with Branch Stalls

If branch frequency is 18% and each branch contributes an average of 1.6 stall cycles (Eq. (5.4)):

$$CPI = 1 + 0.18 \times 1.6 = 1.288. \tag{5.4}$$

This illustrates how even moderate branch penalties raise CPI above the ideal value of 1.

5.5 Cache Simulation Conceptual Model

The cache simulation framework models hierarchy behavior at the level of addresses, tags, and policy events rather than payload data values. A common parameterization is (Eq. (5.5)):

$$(C_1, B, S_1, C_2, S_2, \{\text{MIP,LIP}\}, \{\text{None,+1,Markov,Hybrid}\}, r). \tag{5.5}$$

In this notation, C controls capacity (2^C bytes), B controls block size (2^B bytes), and S controls associativity (2^S ways).

5.6 Hierarchy Semantics in the Model

L1/L2 abstraction: L1 accesses are modeled independently from L2 internals, and L2 requests are modeled as arriving from an upper-level requester. **Write-strategy contrast:** *WBWA* behavior in L1 emphasizes dirty-state tracking and deferred propagation, while *WTWNA* behavior in L2 emphasizes immediate lower-level visibility and no allocation on write misses.

Miss-servicing order effects: ordering choices in read-miss repair and writeback events influence replacement-state evolution and therefore downstream statistics. **Disabled-level interpretation:** a disabled L2 can be represented as an always-miss transit layer with zero L2 hit time in AAT terms.

5.7 Prefetching Concepts

+1 **prefetching:** predicts the next sequential block ($X + 1$) after a demand miss on block X.

Markov prefetching: represents miss-stream transitions as a finite-state table, associates each source miss block with a bounded set of likely successor blocks and frequencies, and selects a prediction from the highest-frequency successor under a deterministic tie-break rule.

Hybrid prefetching: combines correlation-based prediction with sequential fallback when correlation information is absent.

Prefetch usefulness tracking: a prefetch metadata bit distinguishes demand-filled lines from prefetched lines. First demand use of a prefetched line contributes to prefetch-hit interpretation, while eviction before demand use contributes to prefetch-miss interpretation.

5.8 Statistics and AAT Interpretation

Typical hierarchy statistics include access counts, hit/miss counts, hit/miss ratios, writeback counts, and prefetch effectiveness counters. In two-level analysis (Eq. (5.6)):

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot MP_{L1}, \qquad MP_{L1} = AAT_{L2}. \tag{5.6}$$

$$AAT_{L2} = HT_{L2} + MR_{L2,read} \cdot DRAM_TIME. \tag{5.7}$$

$$DRAM_TIME = DRAM_AT + DRAM_AT_PER_WORD \cdot WORDS_PER_BLOCK. \tag{5.8}$$

This form emphasizes that higher-level penalty is the lower-level average access behavior, not a fixed constant.

5.9 Conceptual Tradeoffs Across Parameters

Increasing C generally reduces capacity misses but tends to raise hit time. Increasing S reduces conflict misses but increases lookup complexity. Increasing B can exploit spatial locality but may increase pollution and miss penalty. More aggressive prefetching can reduce compulsory misses yet increase bandwidth pressure and useless fills.

5.10 Chapter 5 Summary

Homework themes, pipeline analysis, and cache simulation models reinforce the same central idea: architecture performance is governed by measurable tradeoffs among latency, throughput, locality, and prediction quality. The simulation concepts provide a concrete way to observe those tradeoffs in controlled workloads. A grouped formula index for the full text is provided in Appendix B.

Chapter 6

Instruction-Level Parallelism and Superscalar Architecture

6.1 Why CPI Can Go Below One

In basic single-issue pipelines, designers often treat $CPI \approx 1$ as a practical lower bound. Superscalar execution challenges that assumption by allowing multiple instructions to make progress in the same cycle. This broader goal is usually described as instruction-level parallelism (ILP). In this context, it is helpful to consider the inverse relationship (Eq. (6.1)):

$$IPC = \frac{1}{CPI}. \tag{6.1}$$

So achieving $CPI < 1$ is exactly equivalent to achieving $IPC > 1$. A useful way to view design limits is that average retire throughput cannot exceed the average throughput of the stages before it (Eq. (6.2)):

$$IPC_{retire} \leq \min(IPC_{fetch},\ IPC_{dispatch},\ IPC_{execute},\ IPC_{state\ update})\,. \tag{6.2}$$

If any major stage can only handle one instruction per cycle on average, then the whole machine cannot retire two instructions per cycle on average, even if another stage is wide enough to do so. In other words, the minimum term in Eq. (6.2) is the throughput bottleneck.

Each term corresponds to a different possible limit. IPC_{fetch} asks how many useful instructions the front end can supply per cycle. $IPC_{dispatch}$ asks how many decoded instructions can be prepared and sent onward. $IPC_{execute}$ asks how many instructions the function units (FUs) and memory system can actually complete. $IPC_{state\ update}$ asks how quickly the machine can accept completed work into precise architectural state. The retire rate cannot exceed the weakest of these rates, because retirement is the final visible outcome of all the earlier pipeline activity.

6.2 Historical Motivation

Historically, supercomputer scalar units demonstrated that out-of-order mechanisms could deliver $IPC > 1$ long before that became mainstream in commodity processors. The core lesson is that pipeline stalls are reduced by combining several mechanisms, not by relying on any single mechanism alone. Instead, they are managed through coordinated fetch, scheduling, execution, and retirement policies. Modern superscalar designs revive this idea by combining hazard management with the ability to issue more instructions per cycle and with stronger speculation.

6.3 Superscalar Pipeline View

A practical superscalar model can be explained with five main logical units plus a final retirement/commit action. Figure 6.1 provides a compact visual summary of how these units interact in the overall datapath. In that figure, the boxed labels denote the main structures or stages, while the smaller labels on the right denote the dominant action on the adjacent arrow bundle: fetch, dispatch, fire or issue, complete, and retire or commit. The front end is drawn as separate Branch Handling and Instruction Fetch boxes, but together they still represent the single logical IF unit described below.

1. Branch Handling and Instruction Fetch (IF) This unit performs instruction fetch (IF) while consulting branch-handling structures such as direction predictors, branch-target mechanisms, and return-address prediction. Its goal is to produce multiple candidate instructions per cycle.

2. Dispatch and Decode (DISP) This unit decodes incoming instructions in program order and dispatches them toward the scheduling structures. Dispatch is where architectural intent is still clearly in order, which is important for precise recovery.

3. Scheduling Scheduling assigns ready instructions to available FUs in time. This is where execution can diverge from program order to exploit parallelism.

4. Execute (EX) FUs perform arithmetic, memory, and control operations with operation-specific latencies. Multiple FUs allow parallel progress.

5. State Update (SU) State update collects completed results, handles squash/recovery, and prepares instructions for precise architectural completion. This unit supports squash/recovery when speculation is wrong. The final outgoing flow from this unit is the retirement or commit action, where results become architecturally visible in program order. In Fig. 6.1, retirement is represented by the final outgoing arrows below State Update rather than by a separate boxed stage.

6.4 Dependency Types and Why They Matter

Superscalar scheduling must respect three dependency classes.

- *RAW* (read-after-write) is a true flow dependency. A later instruction needs a value produced by an earlier one, so executing them in the wrong order would use stale data and produce the wrong result.
- *WAR* (write-after-read) is an anti-dependency. A later instruction must not overwrite a register before an earlier instruction has finished reading it, even though the two instructions do not share a true data value.
- *WAW* (write-after-write) is an output dependency. Two instructions write the same destination, so their writes must appear in the correct order or the final architectural value will be wrong.

Only RAW is fundamentally unavoidable. WAR and WAW are name dependencies that arise from register reuse and can be reduced or removed by renaming the register.

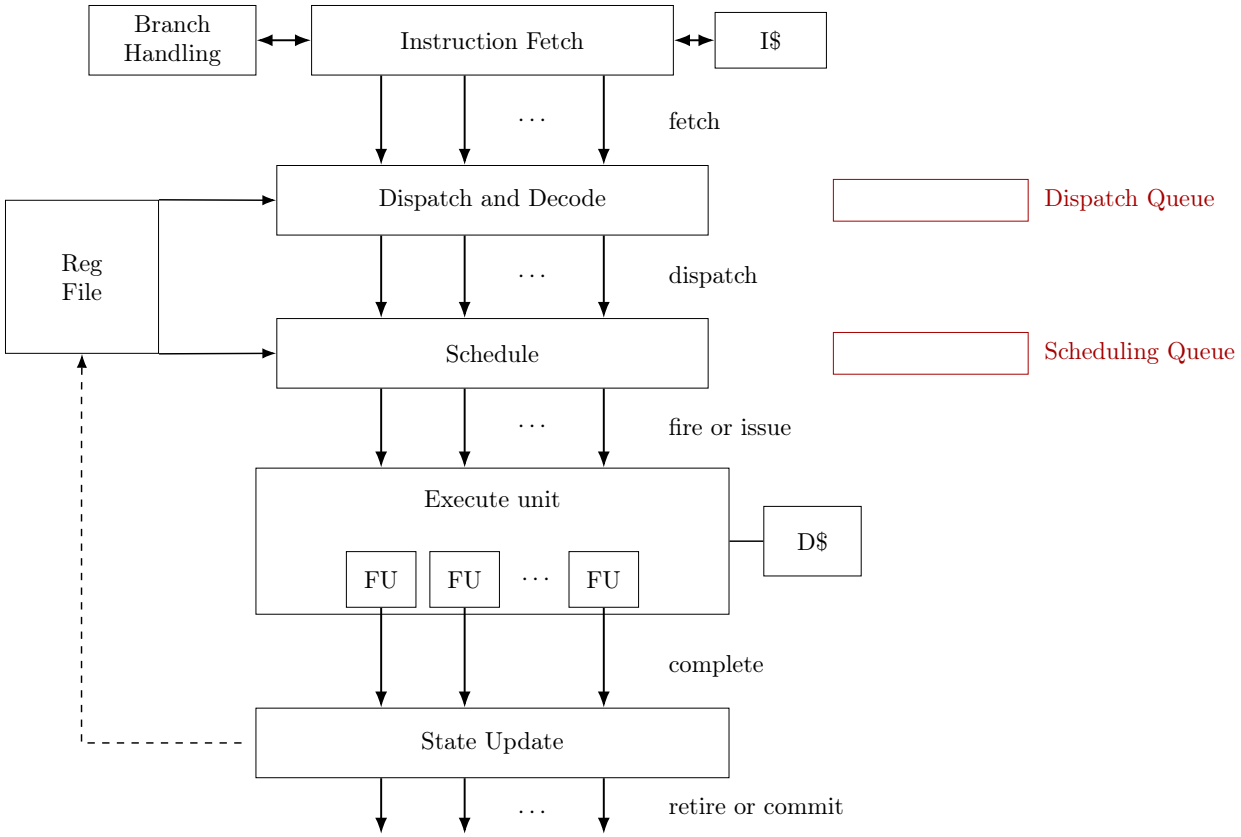

Figure 6.1: Generic logical superscalar pipeline view illustrating one possible organization. It is not itself specific to FICI, FICO, or FOCO; those classifications depend on how firing and completion are implemented. Boxed elements denote the main structures or stages, while the right-side annotations denote the adjacent flow actions: fetch, dispatch, fire or issue, complete, and retire or commit. Representative side structures and queues are also shown.

6.5 Fire/Complete Taxonomy

A compact way to classify designs is by fire order and complete order.

- *FICI*: fire in order, complete in order (classic simple pipeline behavior). Instructions begin execution in program order and also finish in program order, which simplifies control but limits how much parallelism the machine can exploit.

- *FICO*: fire in order, complete out of order (simpler superscalar step). Instructions still begin execution in order, but different latencies allow later instructions to finish first, so some parallel speedup is possible without fully out-of-order scheduling.

- *FOCO*: fire out of order, complete out of order (more aggressive and higher potential ILP). The scheduler may start and finish instructions whenever operands and hardware are ready, which exposes more ILP but requires more machinery to preserve correct architectural behavior.

6.6 Scoreboarding Intuition (FICO Style)

A scoreboard tracks whether sources, destinations, and required function units are currently available. A simplified FICO-style fire condition is shown in Eq. (6.3).

$$\text{Fire if } \neg busy(src_1) \ \wedge \ \neg busy(src_2) \ \wedge \ \neg busy(dest) \ \wedge \ \neg busy(FU_{needed}). \tag{6.3}$$

When an instruction fires, destination and function-unit busy state are set. When the instruction completes, those busy states are cleared according to implementation timing rules. The result is correct execution with modest hardware complexity, but some parallel opportunities are still left unused compared with more aggressive out-of-order firing.

6.7 Function Units and Latency

Superscalar benefit depends on latency balance and available unit count. Common assumptions are short ALU latency, longer multiply latency, and memory latency that depends heavily on cache behavior. Designers therefore optimize both unit availability and scheduling policy, not just raw clock frequency.

6.8 Register Renaming as the Next Step

Renaming introduces additional physical register identities so later writes do not falsely block earlier reads or writes. Conceptually, renaming transforms many WAR/WAW hazards into non-hazards while preserving program meaning. The dependency pressure that remains is primarily RAW, which reflects true data flow. In a typical implementation, the registers named by the ISA are the *architected registers*, while the larger hidden pool used by the machine is the *physical register file*. A *register alias table (RAT)* maps each architected register name to the physical register that currently holds its most recent value. When dispatch renames a destination, it allocates a fresh physical register and updates the RAT so younger instructions will refer to the new location rather than the older one. Ready bits in the physical register file then indicate whether each renamed value has actually been produced yet.

6.9 Tomasulo's Algorithm

Tomasulo's algorithm is a classic out-of-order execution method that allows instructions to fire when their operands are ready rather than strictly in program order. Its central idea is to combine renaming with *reservation stations*, which are scheduling entries that hold an instruction until the required operands and execution resources are available. Historically, this approach is associated with Bob Tomasulo's work on the IBM System/360 Model 91, and it remains the conceptual foundation behind many modern out-of-order schedulers.

Main structures A simplified Tomasulo-style machine contains a dispatch unit, a set of reservation stations, one or more FUs, a result-broadcast network, and a renamed register file. Dispatch still runs in program order, but once instructions occupy reservation stations they may wake up and fire out of order. Each reservation-station entry records the required FU type, the destination tag or physical register, and for each source either a ready value or a tag identifying which older instruction will eventually produce that value.

Dispatch, fire, and complete At dispatch time, each source operand is examined. If the needed value is already ready, the value is copied into the reservation station. If it is not ready, the reservation station records the producing tag instead. The destination is renamed so younger instructions refer to the new producer rather than to the older architectural name. Later, when both sources are ready and a matching FU is available, the instruction can fire even if an older unrelated instruction is still waiting somewhere else in the machine. Completion is a separate event from firing: when an instruction completes, it broadcasts its tag and value on the result bus so dependent instructions can wake up and the renamed register file can be updated.

Illustrative four-instruction trace The following short trace captures the key behavior. Instruction A is a load that misses in the cache. Instruction B is an add that depends on A. Instruction C is an independent load that hits in the cache. Instruction D is an add that depends on C. Because the machine is out of order and the cache is non-blocking, C and then D can make progress while A is still waiting on the miss.

Cycle 0. A, B, C, and D dispatch into reservation stations. Sources that are already available are copied in as values; missing sources are recorded as tags. Destinations are renamed, and the corresponding destination entries are marked not ready.

Cycle 1. A fires, and independent instruction C can also fire because it does not depend on A. This is the key gain from out-of-order scheduling: an older miss does not automatically freeze younger independent work.

Cycles 2–3. Nothing new completes. B is still waiting for A, and D is still waiting for C.

Cycle 4. C completes first and broadcasts its tag and value, for example 420, on the result bus. Reservation stations compare that tag against all waiting operands, so D wakes up and becomes ready to fire.

Cycle 5. D fires and begins its add using the value from C.

Cycle 6. A finally completes its miss and D also wants to complete. With only one result bus, both cannot broadcast at once, so the usual rule is to let the older instruction win. A therefore broadcasts first, for example with value 67, and B immediately becomes ready. D then broadcasts its own result, for example 440, and B can fire as soon as the machine allows post-broadcast wakeup.

This trace shows why it is important to distinguish *fire* from *complete*. Firing sends operands into an FU and begins execution. Completing publishes the result so the rest of the machine can observe it.

Non-blocking caches This Tomasulo-style example also motivates a *non-blocking cache*. Such a cache is often described as supporting *hit under miss*. If A misses in the cache, the machine does not necessarily want every younger memory access to stop. Instead, independent cache accesses may continue to probe the cache pipeline. That allows C to hit and complete while A is still waiting on a lower-level response. That improves throughput, but it also makes control more complex because the cache and scheduler must now track multiple outstanding requests correctly.

6.10 Result Buses and Physical Register Files

Basic Tomasulo has an obvious bottleneck: if all completed instructions must use one common data bus, then only one completion can be broadcast in a cycle. That limitation can be summarized as Eq. (6.4).

$$IPC_{complete} \leq N_{result\ buses}. \tag{6.4}$$

Adding more result buses raises the completion ceiling, but it also increases wakeup complexity because each reservation station must compare against more simultaneous broadcasts.

One historical way to manage this cost is to redesign Tomasulo around an explicit physical register file and a RAT rather than storing all transient values only in reservation-station entries. With physical registers, the scheduler can often replace some associative tag searches with direct lookup by physical register number. That is faster and usually lower power than asking every waiting instruction to compare against every possible producer all the time.

The tradeoff is a heavier register file. If many reservation-station entries are checked each cycle, the physical register file may need substantial read bandwidth. A common first-order estimate is shown in Eq. (6.5).

$$N_{read\ ports} \approx 2N_{sched}, \qquad N_{write\ ports} \approx N_{result\ buses}. \tag{6.5}$$

The factor of two on the read side comes from the common case of two source operands per instruction. Real designs use banking, clustering, bypass paths, operand caching, and other tricks to reduce this raw multi-port cost, but the estimate is a useful baseline for understanding why renamed out-of-order cores are expensive.

6.11 Speculation Without and With Prediction

Superscalar pipelines use speculation to keep FUs busy. Some designs can speculate without high-quality prediction (for example, eager execution ideas), but the useful work fraction is then limited. High-accuracy branch handling usually gives better effective IPC because fewer speculative instructions are squashed.

6.12 Chapter 6 Summary

Instruction-level parallelism turns $CPI < 1$ into a concrete microarchitectural target. Superscalar design achieves this by widening the pipeline's effective throughput and managing dependencies, hazards, and speculative state carefully. FICO-style scoreboarding illustrates the basic mechanism, while renaming, Tomasulo-style wakeup and result broadcast, and stronger out-of-order policies push performance further. These ideas form the bridge from classic five-stage thinking to modern high-throughput scalar cores.

Appendix A

Pipeline Timing-Diagram Interpretation Checklist

A 5-stage timing diagram is best read as a checklist, not as raw symbols. Move cycle by cycle (column by column), then apply the prompts below.

A.1 How to Use This Checklist

Quick Reading Order

1. Mark repeated stage labels (stall behavior).
2. Mark killed instructions (squashes, often shown as dashes).
3. Identify the event that caused each stall/squash (dependence, branch, resource conflict, or memory latency).
4. Compute measured CPI from the traced window.

1. Structural: Resource Organization

1.1 IF and MEM overlap

- **Ask:** Can instruction fetch occur in the same cycle as a load/store data access?
- **Cue:** One cycle shows IF for one instruction while another instruction is in MEM with a real data access.
- **Meaning:** Sustained overlap indicates concurrent instruction/data paths. Repeated IF stalls when load/store reaches MEM suggests a shared bottleneck.

1.2 Single-issue vs superscalar

- **Ask:** Is the machine *single-issue* or *superscalar* (*multi-issue*)?
- **Cue:** In simple single-issue diagrams, each stage has at most one instruction per cycle.
- **Meaning:** Two instructions shown in one stage in one cycle (for example two EX uses) implies multi-issue and/or multiple functional units.

1.3 Multi-cycle execute units

- **Ask:** Do some operations occupy EX for multiple cycles?
- **Cue:** One instruction appears in EX across consecutive cycles (EX, EX, EX, ...).
- **Meaning:** That operation is using a multi-cycle execute unit and can back up younger instructions.

2. Data: Dependence and Forwarding

2.1 RAW hazard location and cost

- **Ask:** Where are *RAW* hazards, and how many bubbles do they add?
- **Cue:** Consumer instruction repeats a stage (usually ID, sometimes IF) while waiting for producer timing.
- **Meaning:** Number of repeated cells is the local dependence penalty.

2.2 Forwarding strength

- **Ask:** Is forwarding present, and from where?
- **Cue:** Back-to-back ALU dependencies proceed without stalling even though producer WB occurs later.
- **Meaning:** This usually implies EX/MEM→EX and MEM/WB→EX forwarding. If simple ALU→ALU pairs stall, forwarding is limited.

2.3 Load-use behavior

- **Ask:** What is the load-use penalty?
- **Cue:** Load immediately followed by consumer, where consumer stalls one cycle.
- **Meaning:** Classic 5-stage with forwarding still needs a one-cycle load-use stall because load data appears at end of MEM.

2.4 WB/ID same-cycle timing

- **Ask:** Can ID read a register in the same cycle an older instruction writes it during the WB stage?
- **Cue:** Consumer ID aligns with producer WB, with no extra bubble.
- **Meaning:** Implies write-before-read register timing and/or WB→ID bypass behavior.

3. Control: Branch and Redirection Policy

3.1 Resolution stage and penalty

- **Ask:** Where is branch outcome resolved, and what is the visible penalty?
- **Cue:** Find where target/direction becomes known; then count wrong-path squashes or inserted bubbles before correct-path flow stabilizes.
- **Meaning:** Later resolution generally increases misfetch cost.

3.2 Delay slot vs squash slot

- **Ask:** Does the post-branch slot always execute, or can it be killed?
- **Cue:** Slot instruction either always completes (*delay slot*) or is conditionally killed (*squash slot*/branch-likely style).
- **Meaning:** Always-executed slot indicates delay-slot semantics; conditional kill indicates squash-style semantics.

3.3 Taken/not-taken inference (branch-likely style)

- **Ask:** Which branch outcome occurred?
- **Cue:** Whether the slot instruction executes or is squashed.
- **Meaning:** In this text's branch-likely examples, a squashed slot corresponds to not taken.

3.4 Prediction vs no prediction

- **Ask:** Does fetch speculate, and is speculation often correct?
- **Cue:** Continued fetching past unresolved branches followed by occasional squashes (speculation), versus front-end waiting for resolution (no speculation).
- **Meaning:** Wrong-path fetches later killed imply predictive/fixed speculation; repeated front-end pauses imply non-speculative handling.

4. Memory: Miss Effects in the Timeline

4.1 Miss penalty visibility

- **Ask:** Are long memory latencies visible?
- **Cue:** Load/store occupies MEM for multiple cycles (M, M, M, . . .), with younger instructions backing up.
- **Meaning:** Indicates cache-miss (or similar long-latency memory) behavior.

4.2 Blocking vs non-blocking cache behavior

- **Ask:** Is cache behavior *blocking*, or can it do *hit-under-miss* (*non-blocking*)?
- **Cue:** Either all younger work freezes behind a miss, or independent work continues.
- **Meaning:** Full freeze implies blocking behavior (common in simple in-order models). Continued independent progress suggests non-blocking support and/or more advanced scheduling.

5. Performance from the Diagram

5.1 Measured CPI of a trace window If a window spans C cycles and retires N_{ret} non-squashed instructions (Eq. (A.1)):

$$\mathrm{CPI}_{measured} = \frac{C}{N_{ret}}. \tag{A.1}$$

Squashed instructions consume cycles but do not count as retired instructions.

5.2 Stall-cycle attribution by cause Assign repeated-stage cells and squashes to causes:

- dependence waiting (RAW/load-use),
- control redirection (branch squashes/bubbles),
- structural conflicts (resource contention),
- memory-latency extension (repeated MEM).

This gives a practical CPI breakdown from one timing chart.

6. Common Gotchas

- ALU instructions may still appear in MEM in pedagogical 5-stage diagrams; this alone does not prove a data-memory access.
- IF stalls can be secondary backpressure from a downstream stall (often ID), not a fetch-unit limitation by itself.
- Squash and stall are different: squashed instructions are killed; stalled instructions are preserved in place with repeated stage labels.

Appendix B

Formula Compendium (Grouped and Labeled)

This appendix gathers the core formulas used across Chapters 1–6.

Chapter 1 Formula Set

Chapter 1-F1 (Vector Serial vs Pipelined Cycles)

$$\text{serial cycles} = nk, \qquad \text{pipeline cycles} \approx k + n - 1. \tag{B.1}$$

Chapter 2 Formula Set

Chapter 2-F1 (CPU Time Decomposition)

$$\text{CPU time} = \text{IC} \times \text{CPI} \times \text{CT}. \tag{B.2}$$

Chapter 2-F2 (Frequency Form)

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}}{f}. \tag{B.3}$$

Chapter 2-F3 (Instruction-Mix Total Cycles)

$$\text{Total cycles} = \sum_i \left(\text{IC}_i \times \text{CPI}_i\right). \tag{B.4}$$

Chapter 2-F4 (Average CPI)

$$\text{CPI}_{avg} = \frac{\text{Total cycles}}{\text{Total instructions}}. \tag{B.5}$$

Chapter 2-F5 (Speedup Definition)

$$S = \frac{T_{old}}{T_{new}}. \tag{B.6}$$

Chapter 2-F6 (Amdahl's Law)

$$S_{Amdahl} = \frac{1}{(1-f) + \frac{f}{s}}. \tag{B.7}$$

Chapter 2-F7 (Amdahl Upper Bound)

$$S_{\text{upper}} = \frac{1}{1-f}. \tag{B.8}$$

Chapter 2-F8 (Gustafson's Law)

$$S_{Gustafson} = \alpha + (1-\alpha)N. \tag{B.9}$$

Chapter 2-F9 (Arithmetic Mean for Time)

$$\bar{T}_{arith} = \frac{1}{n}\sum_{i=1}^{n} T_i. \tag{B.10}$$

Chapter 2-F10 (Harmonic Mean for Rate)

$$\bar{R}_{harm} = \frac{n}{\sum_{i=1}^{n} \frac{1}{R_i}}. \tag{B.11}$$

Chapter 3 Formula Set

Chapter 3-F1 ((C, B, S) Definitions)

$$\text{capacity} = 2^C \text{ bytes}, \qquad \text{block size} = 2^B \text{ bytes}, \qquad \text{associativity} = 2^S \text{ ways}. \tag{B.12}$$

Chapter 3-F2 (Total Blocks and Sets)

$$\text{blocks} = 2^{C-B}, \qquad \text{sets} = \frac{2^C}{2^B \cdot 2^S} = 2^{C-B-S}. \tag{B.13}$$

$$\text{lines} = \text{sets} \cdot \text{ways} = 2^{C-B-S} \cdot 2^S = 2^{C-B}. \tag{B.14}$$

Direct-mapped corollary: if $S = 0$, then ways $= 1$ and lines $=$ sets. Set-associative corollary: if $S > 0$, then lines $>$ sets.

Chapter 3-F3 (Address-Field Widths, General AW)

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = AW - (C - S). \tag{B.15}$$

$$\text{Address} = [\text{Tag} \mid \text{Index} \mid \text{Offset}]. \tag{B.16}$$

Chapter 3-F4 (Address-Field Widths, 32-bit Address)

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = 32 - (C - S) = 32 - C + S. \tag{B.17}$$

Chapter 3-F5 (Way Selection Relationship)

$$\text{ways} = 2^S, \qquad \log_2(\text{ways}) = S. \tag{B.18}$$

Chapter 3-F6 (Single-Level Average Access Time)

$$AAT = HT + MR \cdot MP. \tag{B.19}$$

Chapter 3-F7 (Bit Overhead from Valid/Dirty)

$$\text{overhead fraction} = \frac{n_v + n_d}{2^{B+3}}, \qquad n_v = 1, \quad n_d \in \{0, 1\}. \tag{B.20}$$

Chapter 3-F8 (Full-Block Miss Penalty Model)

$$MP_{full} = T_1 + (N_{sb} - 1)T_f, \qquad N_{sb} = 2^B/W. \tag{B.21}$$

Chapter 3-F9 (Early-Restart Approximation)

$$MP_{ER} \approx T_1 + \frac{N_{sb} - 1}{2} T_f. \tag{B.22}$$

Chapter 3-F10 (Recursive Two-Level Form)

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot AAT_{L2}. \tag{B.23}$$

Chapter 3-F11 (VIPT Constraint)

$$C - S \leq P. \tag{B.24}$$

Chapter 4 Formula Set

Chapter 4-F1 (Pipelined Clock Approximation)

$$T_{clk,pipe} \approx T_s + T_{reg}. \tag{B.25}$$

Chapter 4-F2 (Ideal Long-Run Pipeline Speedup)

$$S_{ideal} \approx \frac{NT_s}{T_s + T_{reg}}. \tag{B.26}$$

Chapter 4-F3 (Finite-Length Pipelined Time)

$$T_{pipe} \approx (N + M - 1)T_{clk,pipe}. \tag{B.27}$$

Chapter 4-F4 (Unpipelined Time for M Instructions)

$$T_{unpiped} \approx MNT_s. \tag{B.28}$$

Chapter 4-F5 (Practical CPI Model)

$$CPI = 1 + \text{stalls}_{avg}. \tag{B.29}$$

Chapter 4-F6 (Branch-Stall CPI Contribution)

$$CPI \approx 1 + f_{branch} \cdot \text{stall}_{branch,avg}. \tag{B.30}$$

Chapter 5 and Simulation Formula Set

Chapter 5-F1 (Two-Level AAT with Explicit L1 Penalty)

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot MP_{L1}, \qquad MP_{L1} = AAT_{L2}. \tag{B.31}$$

Chapter 5-F2 (L2 AAT Model)

$$AAT_{L2} = HT_{L2} + MR_{L2,read} \cdot DRAM_TIME. \tag{B.32}$$

Chapter 5-F3 (DRAM Block Service Model)

$$DRAM_TIME = DRAM_AT + DRAM_AT_PER_WORD \cdot WORDS_PER_BLOCK. \tag{B.33}$$

Chapter 6 Formula Set (ILP and Superscalar)

Chapter 6-F1 (IPC and CPI Relationship)

$$IPC = \frac{1}{CPI}. \tag{B.34}$$

Chapter 6-F2 (CPI/IPC Threshold Equivalence)

$$CPI < 1 \iff IPC > 1. \tag{B.35}$$

Chapter 6-F3 (Retire-Throughput Bound)

$$IPC_{retire} \leq \min(IPC_{fetch},\ IPC_{dispatch},\ IPC_{execute},\ IPC_{state\ update}). \tag{B.36}$$

Chapter 6-F4 (Simplified Scoreboard Fire Condition)

$$\text{Fire if } \neg busy(src_1) \wedge \neg busy(src_2) \wedge \neg busy(dest) \wedge \neg busy(FU_{needed}). \tag{B.37}$$

Chapter 6-F5 (Result-Bus Completion Bound)

$$IPC_{complete} \leq N_{result\ buses}. \tag{B.38}$$

Chapter 6-F6 (Physical Register File Port Estimate)

$$N_{read\ ports} \approx 2N_{sched}, \qquad N_{write\ ports} \approx N_{result\ buses}. \tag{B.39}$$

Appendix C

Glossary of Core Definitions

This appendix collects first-pass definitions used throughout Chapters 1–6.

Parallelism Doing multiple useful operations concurrently to increase throughput.

Speculation Executing work before all required conditions are confirmed, with recovery if assumptions are wrong.

Prediction Estimating uncertain future behavior to guide speculative execution.

Instruction-Level Parallelism (ILP) Overlap of independent instructions within one thread.

Thread-Level Parallelism (TLP) Concurrent execution of multiple threads on one or more cores.

SIMD One instruction stream operating on multiple data elements in lockstep.

MIMD Multiple instruction streams operating on multiple data streams independently.

Pipelining Temporal parallelism where different stages process different instructions simultaneously.

Superscalar (Multi-Issue) Pipeline organization that can issue more than one instruction per cycle.

Tomasulo's Algorithm Out-of-order scheduling method that combines renaming, reservation stations, and result broadcast to wake dependent instructions when operands become ready.

Reservation Station Scheduling entry that holds an instruction, its operand state, and dependency tags until it can execute.

Common Data Bus (CDB) Shared result-broadcast path used to distribute a completed value and its producer tag to waiting instructions.

Result Bus Generalized form of the CDB; modern designs often use more than one such broadcast path.

Architected Register Register name defined by the ISA and visible to software.

Physical Register Internal hardware register used to hold renamed values beyond the architected register set.

Register Alias Table (RAT) Mapping table from architected register names to the current physical register locations.

Ready Bit Status bit indicating that a register or operand value has already been produced and may be consumed.

Runtime Time to complete one job (response/turnaround time).

Throughput Completed work per unit time.

Amdahl's Law Speedup limit for fixed-size workloads when only a fraction is accelerated.

Gustafson's Law Scaled-speedup model where workload size grows with processor count.

Average Access Time (AAT) Expected memory-access latency, typically modeled as hit time plus miss rate times miss penalty.

SRAM Fast, low-latency, higher-area memory used for on-chip caches.

DRAM Dense, lower-cost, higher-latency memory used for main memory capacity.

Hit Time (HT) Latency to serve an access from the current cache level.

Miss Ratio (MR) Fraction of accesses at a level that miss.

Miss Penalty (MP) Extra latency to service a miss from lower levels.

Compulsory Miss First-reference miss for data not previously loaded.

Capacity Miss Miss caused by working set exceeding effective cache capacity.

Conflict Miss Miss caused by mapping/placement constraints despite sufficient total capacity.

Write-Back Write policy where modified data is propagated on eviction.

Write-Through Write policy where lower level is updated on each write hit.

Write-Allocate Write-miss policy that fetches and installs the block before write.

Write-No-Allocate Write-miss policy that bypasses install and writes lower level directly.

PIPT Physically indexed, physically tagged cache organization in which TLB translation typically precedes L1 indexing/tag lookup.

VIPT Virtually indexed, physically tagged cache organization that overlaps indexing with translation.

TLB Translation lookaside buffer storing recent virtual-to-physical translations.

Structural Hazard Stall risk from insufficient hardware resources.

Data Hazard Stall risk from operand read/write ordering conflicts.

Control Hazard Stall risk from unresolved branch/jump control flow.

RAW Read-after-write dependence hazard.

WAR Write-after-read dependence hazard.

WAW Write-after-write dependence hazard.

Forwarding (Bypassing) Sending a produced value directly to a consumer before register write-back.

Branch Target Buffer (BTB) Predictor structure that supplies likely branch targets early in fetch.

Aliasing Predictor or cache entry interference from unrelated addresses mapping to the same slot.

Return-Address Stack (RAS) Small stack predictor for call/return target addresses.

Direction Prediction Branch predictor decision for taken versus not taken.

BTFNT Backward taken, forward not taken static branch heuristic.

Delay Slot ISA semantic in which the instruction after a branch executes before control transfer takes effect.

Squash Slot Post-branch instruction position that may be conditionally killed based on branch outcome/policy.

WBWA Write-back, write-allocate pairing.

WTWNA Write-through, write-no-allocate pairing.

Blocking Cache Cache behavior where an outstanding miss prevents servicing younger accesses.

Non-Blocking Cache Cache behavior allowing selected younger accesses to proceed during a miss.

Hit-Under-Miss Ability to serve cache hits while a miss is still outstanding.